

01 - Nueva tabla y su CRUD

Ejercicio 01 — Nueva tabla y su CRUD

Foco: backend, generación guiada por un patrón existente **Tiempo estimado:** 1 clase [↩ Volver al README](#)

Objetivo

Que aprendas a usar la IA para **generar código nuevo que respete un patrón que ya existe** en el proyecto. Este es el uso más simple y más seguro de la IA ( habilitado): "mirá cómo está hecho esto, hacelo igual para esto otro".

El problema

El proyecto tiene dos entidades: `alumnos` y `cursos`. Cada una tiene su tríada completa:

- `src/controllers/alumnos-controller.js`
- `src/services/alumnos-service.js`
- `src/repositories/alumnos-repository.js`

(y lo mismo para `cursos`).

Queremos agregar una **tabla nueva:** `materias`. Tiene que quedar enchufada igual que las otras dos, expuesta en `/api/materias` con los 5 endpoints CRUD.

Aprovechamos para dejar creada también la tabla `calificaciones`, que relaciona `alumnos` con `materias`. En **este** ejercicio solo hacés el CRUD de `materias` (simple, parecido a `cursos`). La tabla `calificaciones` es más rica —tiene FKs, una restricción de unicidad y reglas de negocio (notas de 0 a 10, no duplicar alumno+materia, JOINS para traer nombres)— y te va a servir como entidad de práctica en los ejercicios siguientes (validaciones en el **04**, performance y JOINS en el **08**).

Script SQL (creá las dos tablas)

```
-- Tabla materias
CREATE TABLE materias (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(75) NOT NULL
);
```

```
-- Tabla calificaciones
-- Cada alumno tiene UNA sola calificación por materia (no se repite la combinación
  alumno+materia).

CREATE TABLE calificaciones (
  id SERIAL PRIMARY KEY,
  id_alumno INT NOT NULL REFERENCES alumnos(id),
  id_materia INT NOT NULL REFERENCES materias(id),
  nota INT NOT NULL,
  fecha DATE NOT NULL DEFAULT CURRENT_DATE,
  UNIQUE(id_alumno, id_materia)
);

-- Datos de prueba para materias
INSERT INTO materias (nombre) VALUES ('Matemática');
INSERT INTO materias (nombre) VALUES ('Lengua');
INSERT INTO materias (nombre) VALUES ('Historia');
INSERT INTO materias (nombre) VALUES ('Programación');
INSERT INTO materias (nombre) VALUES ('Base de Datos');
```

Las calificaciones no se siembran: las vas a crear desde la API (POST) cuando llegues a los ejercicios que usan esa tabla.

Qué tenés que lograr

1. Ejecutar el script SQL de arriba para crear `materias` y `calificaciones` (+ los datos de prueba de `materias`).
2. `materias-repository.js`, `materias-service.js`, `materias-controller.js` siguiendo **exactamente** el patrón de `cursos` (que es el CRUD más simple del proyecto).
3. Registrar el controller en `src/server.js` (`app.use("/api/materias", ...)`).
4. Probar los 5 endpoints con Postman y exportar la collection.

Cómo encarar el prompting

 **Trampa:** si le pedís "hacé el CRUD de materias" sin pasarle el código existente, la IA te va a inventar un estilo distinto (probablemente con un ORM, o con otra forma de manejar el Pool). Después no va a encajar y vas a perder más tiempo arreglándolo que escribiéndolo vos.

Tu prompt tiene que incluir, mínimo:

- **Rol:** desarrollador backend Node/Express.
- **Contexto:** arquitectura en capas, `pg` sin ORM, ES modules, y **pegar** `cursos-repository.js`, `cursos-service.js` y `db-pg.js` **como referencia de estilo** (`cursos` es el CRUD más parecido al de `materias`).
- **Tarea:** generar las 3 capas para `materias`.

- **Restricciones:** sin dependencias nuevas, mismo estilo (clases, delegar el acceso a datos en la clase `DbPg` con `this.db.queryAll/queryOne/queryReturnId/queryRowCount`, queries `$1`), mantener los `console.log`.
- **Iteración:** pedir primero el repository, revisarlo, después el service y el controller.

💡 **Tip:** Generá **de a una capa**. Si pedís las 3 juntas es más difícil revisar y la IA tiende a tomar atajos.
Repository → la revisás → service → la revisás → controller.

🔍 Verificación del resultado (¿la IA lo hizo bien?)

Marcá cada ítem cuando lo verifiques **vos** (no la IA):

- ☐ El repository delega el acceso a datos en la clase `DbPg` (`this.db.queryAll/queryOne/...`), igual que `alumnos-repository.js`. No toca el `Pool` directamente ni crea un `Client` nuevo por request (de eso se encarga `db-pg.js` con lazy init).
- ☐ Las queries usan placeholders `$1`, `$2...` (no concatenación de strings → eso sería SQL injection, ver ejercicio 09).
- ☐ El controller devuelve los **status codes correctos**: 200, 201 en POST, 404 cuando no existe, 400 en error de input.
- ☐ El `update` valida que el `id` de la URL coincida con el del body (mirá cómo lo hace `alumnos-controller.js` en el `PUT`).
- ☐ El controller está registrado en `server.js` y los 5 endpoints responden en Postman.
- ☐ No aparecieron dependencias nuevas en `package.json`.

🤔 Pregunta para el oral: en el `createAsync` de `alumnos`, ¿por qué se usa `?? ''` y no `|| ''`? ¿Qué pasaría con un alumno que `hace_deportes = false`? (Pista: fijate en `alumnos-repository.js` cómo se arma el array `values` con `?? ''` y `?? 0`, y pensá qué hace `||` cuando el valor es `0` o `false`.)

✅ Entrega

Completá la [plantilla de bitácora](#) para este ejercicio. Acordate de:

- Marcar en el código con `// [IA]` y `// [YO]`.
- Adjuntar el historial de prompts.
- En la reflexión, contá **qué cambiaste** del código que te dio la IA (siempre hay algo).